

Name Rahul Rajput

Standard 4 Section D Roll No. 18F24CS239

Subject ADA observation (lab)

Sl.No.	Date	Title	Page No	Teacher's Sign
1.	11 March	Merge Sort Technique	1 to 4, 6	H
2.	11 March	Leetcode - 26	5, 6	
3.	18 March	Quick Sort Technique	7 to 9	
4.	18 March	Leetcode - 21	10-11	H
5.	25 March	Prim's Algorithm	12-15	
6.	25 March	Kruskal's Algorithm	16-19	
7.	1 April	Dijkstra's Algorithm	20-23	
8.	1 April	Leetcode 1928	24-26	H
9.	15 April	Fractional Knapsack	27-31	
10.	15 April	Leetcode 922	32	
11.	22 April	Floyd's Algorithm	31-33	H
12.	22 April	Leetcode 1334	34-35	
13.	6 May	0/1 Knapsack using DP	36-39	
14.	6 May	Leetcode 638	40-42	H
15.	27 May	Topological Ordering	43-46	
16.	27 May	N-Queens problem	47-49	
17.	27 May	Johnson Trotter (Leetcode)	50-53	H
18.	3 June	Heap Sort	54-56	
				9/6/26

- ① Sort a given set of N integer elements using Merge Sort Technique and compute its time taken to sort.

Pseudocode

MergeSort (array A , left, right)

if $\text{left} < \text{right}$

mid = floor $((\text{left} + \text{right}) / 2)$

MergeSort (A , left, mid)

MergeSort (A , mid+1, right)

Merge (A , left, mid, right)

merge (A , left, mid, right)

$n1 = \text{mid} - \text{left} + 1$

$n2 = \text{right} - \text{mid}$

let leftArray $[0 \dots n1-1]$

let rightArray $[0 \dots n2-1]$

for $i = 0$ to $n1-1$

leftArray $[i] = A[\text{left} + i]$

for $j = 0$ to $n2-1$

rightArray $[j] = A[\text{mid} + 1 + j]$

$i = 0$

$j = 0$

$k = \text{left}$

while $i < n1$ & $j < n2$

if leftArray $[i] \leq$ rightArray $[j]$

$A[k] = \text{leftArray}[i]$

$i = i + 1$

else

$A[k] = \text{rightArray}[j]$

$j = j + 1$

$k = k + 1$

while $i < n1$

$A[k] = \text{leftArray}[i]$

$i = i + 1$

$k = k + 1$

while $j < n2$

$A[k] = \text{rightArray}[j]$

$j = j + 1$

$k = k + 1$

②

Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void merge (int arr[], int left, int mid, int right) {
```

```
    int i, j, k;
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    int *L = (int *) malloc (n1 * sizeof
```

```
#include <stdio.h>
```

```
void merge (int arr[], int l, int m, int r) {
```

```
    int i, j, k;
```

```
    int n1 = m - l + 1;
```

```
    int n2 = r - m;
```

```
    int L[n1], R[n2];
```

```
    for (i = 0; i < n1; i++) {
```

```
        L[i] = arr[l + i];
```

```
    }
```

```
    for (j = 0; j < n2; j++) {
```

```
        R[j] = arr[m + 1 + j];
```

```
    }
```

```
    i = 0;
```

```
    j = 0;
```

```
    k = l;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) {
```

```
            arr[k] = L[i];
```

```
            i++;
```

```
        } else {
```

```
            arr[k] = R[j];
```

```
            j++;
```

```
        }
```

```
        k++;
```

```
    }
```

```
    while (i < n1) {
```

```
        arr[k] = L[i];
```

```
        i++;
```

```
        k++;
```

```
    }
```

while (j < n) {

arr[j] = arr[i];

j++;

i++;

}

}

void mergeSort (int arr[], int l, int r) {

if (l < r) {

int m = (l + r) / 2;

mergeSort (arr, l, m);

mergeSort (arr, m + 1, r);

merge (arr, l, m, r);

}

}

int main() {

int n, i;

printf ("Enter number of elements : ");

scanf ("%d", &n);

int arr[n];

printf ("Enter %d elements : \n", n);

for (i = 0; i < n; i++) {

scanf ("%d", &arr[i]);

}

mergeSort (arr, 0, n - 1);

printf ("Sorted array : \n");

for (i = 0; i < n; i++) {

printf ("%d", arr[i]);

}

return 0;

}

O/p :

Enter the number of elements : 5

Enter 5 elements : 49

64

55

32

4

Sorted array : 4 32 49 55 64

(4)

Time complexity for merge sort

Recurrence relation for ms

$$T(n) = 2T(n/2) + n$$

Substitute $T(n/2)$:

$$\begin{cases} \text{merge} - n \\ \text{mergesort} - 2T(n/2) \end{cases}$$

$$T(n) = 2 [2T(n/4) + n/2] + n$$

$$= 4T(n/4) + n + n$$

$$= 4T(n/4) + 2n$$

Again substituting,

$$T(n) = 8T(n/8) + 3n$$

After k substitutions,

$$T(n) = 2^k T(n/2^k) + kn \quad \text{--- (1)}$$

Recursion stops when

$$n/2^k = 1$$

$$n = 2^k$$

$$\log_2 n = \log_2 2^k$$

$$k = \log_2 n \quad \text{--- (2)}$$

Substituting k in (1),

$$T(n) = nT(1) + n \log n$$

$$T(n) = n + n \log n$$

$$T(n) = O(n \log n)$$

LEETCODE - 26

Given an integer array `nums` sorted in non-decreasing order, remove the duplicates in place such that each unique element should be kept the same.

Consider the number of unique elements in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`. The first `k` elements of `nums` should contain the unique numbers in sorted order. The remaining elements beyond index `k-1` can be ignored.

Example:

Input: `nums = [1,1,2]`

Output: `nums = [1,2,-]`

Explanation:

Your function should return `k=2`, with the first two elements of `nums` being 1 & 2 respectively. It doesn't matter what you leave beyond the returned `k` (hence they are underscores).

Code -

```
#include <stdio.h>
int removeDuplicates (int * nums, int numsSize) {
    if (numsSize == 0) return 0;
    int i = 0;
    for (int j = 1; j < numsSize; j++) {
        if (nums[j] != nums[i]) {
            i++;
            nums[i] = nums[j];
        }
    }
    return i+1;
}
```

⑥

Program Output Leetcode:

case 1

nums = [1, 1, 2]

o/p

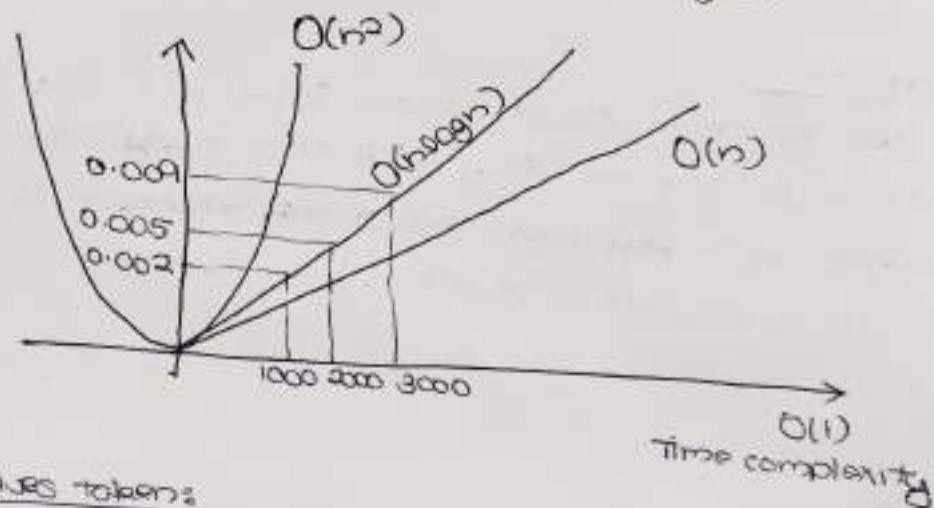
[1, 2]

Expected

[1, 2]

Graph (Program 1)

Time complexity for merge sort : $O(n \log n)$



Values taken:

1000	0.002
2000	0.005
3000	0.009
4000	0.015
5000	0.023

Q2 Write a C program for Quick Sort

Pseudo
code

Quicksort (array A, low, high)

if low < high

pi = Partition (A, low, pi-1)

quicksort (A, low, pi-1)

quicksort (A, pi+1, high)

Partition (A, low, high)

pivot = A[high]

i = low-1

for j = low to high-1

if A[j] <= pivot

i = i+1

swap A[i] with A[j]

swap A[i+1] with A[high]

return i+1

swap (x, y)

temp ← x

x ← y

y ← temp

Code

void swap (int arr[], int i, int j) {

int temp = arr[i];

arr[i] = arr[j];

arr[j] = temp;

}

int partition (int arr[], int low, int high) {

int pivot = arr[high];

int i = low-1;

int j;

for (j = low; j < high; j++) {

if (arr[j] <= pivot) {

i++;

8

swap (arr, i, j);

}

}

swap (arr, i+1, high);

return i+1;

}

void quicksort (int arr[], int low, int high) {

if (low < high) {

int pi = partition (arr, low, high);

quicksort (arr, low, pi-1);

quicksort (arr, pi+1, high);

}

}

int main () {

int n;

printf ("Enter number of elements: ");

scanf ("%d", &n);

if (n <= 0) {

printf ("Please enter positive number");

return 1;

}

int arr[n];

printf ("Enter %d elements", n);

for (int i = 0; i < n; i++) {

scanf ("%d", &arr[i]);

}

printf ("\n");

quicksort (arr, 0, n-1);

printf ("Sorted array: ");

for (int i = 0; i < n; i++) {

printf ("%d ", arr[i]);

}

printf ("\n");

}

return 0;

Output :

9

Enter number of elements : 5

Enter 5 numbers :

48 12 65 23 7

Sorted array :

7 12 23 48 65

Time complexity for Quick sort :

Recurrence relation for QS :

$$T(n) = 2T(n/2) + O(n)$$

$$T(n) = 2^k T(n/2^k) + kO(n) \quad \text{--- (1)}$$

Recursion stops when

$$n/2^k = 1$$

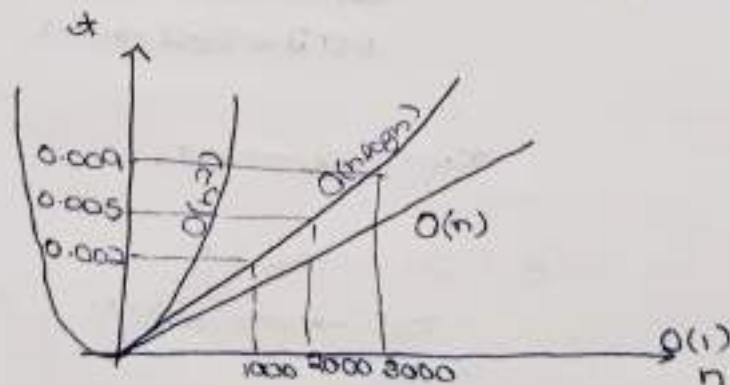
$$\Rightarrow k = \log_2 n$$

$$\begin{aligned} T(n) &= 2^{\log_2 n} T(1) + k \log n \\ &= nT(1) + n \log n \end{aligned}$$

$$T(n) = O(n \log n)$$

Graph :

Time complexity for quick sort = $O(n \log(n))$



Values taken :

1000	0.002
2000	0.005
3000	0.009
4000	0.015
5000	0.023

H, 8/3/26

⑥

LEETCODE: 21

You are given the heads of two sorted linked lists list1 & list2. merge the two lists into one sorted list. the list should be made by splicing together the nodes of the first two lists. Return the head of the merged linked list.

Code:

```
#include <stdio.h>

struct ListNode {
    int val;
    struct ListNode *next;
};

struct ListNode *mergeTwoLists(struct ListNode *list1,
                                struct ListNode *list2) {

    struct ListNode dummy;
    struct ListNode *tail = &dummy;
    dummy.next = NULL;

    while (list1 != NULL && list2 != NULL) {
        if (list1->val < list2->val) {
            tail->next = list1;
            list1 = list1->next;
        } else {
            tail->next = list2;
            list2 = list2->next;
        }
        tail = tail->next;
    }
    if (list1 != NULL) {
        tail->next = list1;
    } else {
        tail->next = list2;
    }
    return dummy.next;
}
```

Output:

⑪

Input:

list1 = [1, 2, 4]

list2 = [1, 3, 4]

Output:

[1, 1, 2, 3, 4, 4]

Expected:

[1, 1, 2, 3, 4, 4]

/ *St*
18/8/22

Q. Write a C program to demonstrate Prim's Algorithm.

Pseudocode :

PRIM (G, start):

Input: Graph $G(V, E)$, start, vertex

Output: Minimum Spanning tree (MST)

create a set visited = $\emptyset$

create a min priority queue PQ

for each vertex v in V :

$\text{key}[v] = \infty$

$\text{parent}[v] = \text{NIL}$

$\text{key}[\text{start}] = 0$

insert $(0, \text{start})$ into PQ

while PQ is not empty:

$(\text{current_key}, u) = \text{extract_min}()$

 if u is already in visited:

 continue

 add u to visited

 for each neighbour v of u :

$\text{weight} = \text{weight}(u, v)$

 if v not in visited AND $\text{weight} < \text{key}[v]$

$\text{key}[v] = \text{weight}$

$\text{parent}[v] = u$

 insert $(\text{key}[v], v)$ into PQ

return parent



Code:

```
#include <stdio.h>
#include <limits.h>
#define MAX 100

int n;

int minkey (int key[], int mstSet[]) {
    int min = INT_MAX, min_index = -1;
    for (int v = 0; v < n; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

void printMST (int parent[], int graph[MAX][MAX]) {
    printf ("Edge \t Weight \n");
    for (int i = 1; i < n; i++) {
        printf ("%d \t %d \t %d \n", parent[i], i,
            graph[i][parent[i]]);
    }
}

void primMST (int graph[MAX][MAX]) {
    int parent[MAX];
    int key[MAX];
    int mstSet[MAX];

    for (int i = 0; i < n; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < n-1; count++) {
        int u = minkey (key, mstSet);
        mstSet[u] = 1;
```

(14)

```

for (int v = 0; v < n; v++) {
    if (graph[u][v] == 0 && mstSet[v] == 0 &&
        graph[u][v] < key[v]) {
        parent[v] = u;
        key[v] = graph[u][v];
    }
}
}
}
printMST (parent, graph);
}

```

```

int main () {
    int graph[MAX][MAX];
    printf ("Enter number of vertices");
    scanf ("%d", &n);
    printf ("Enter adjacency matrix.\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf ("%d", &graph[i][j]);
        }
    }
    printMST (graph);
    return 0;
}

```

OUTPUT :

Enter number of vertices : 4

Enter adjacency matrix :

```

0 2 0 5
2 0 3 8
0 3 0 0
6 8 0 0

```

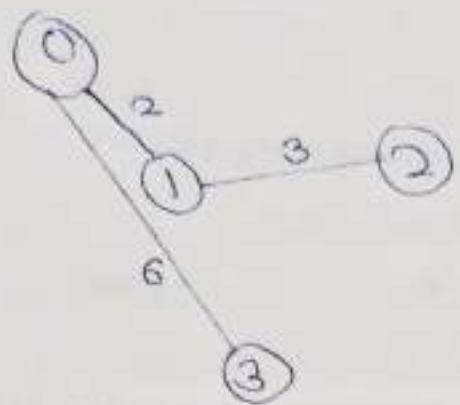
O/p =>

Edge	Weight
0-1	2
1-2	3
0-3	6

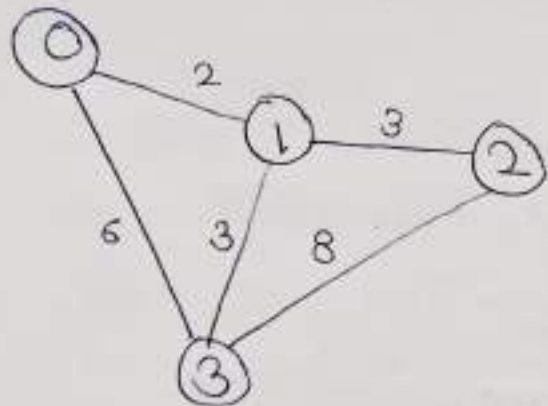
Graph:

Time complexity: $O(V^2)$

mst 3



ex-graph 3



⑩ Write a C program to demonstrate Kruskal's Algorithm

Pseudocode:

KRUSKAL(G):

Input: Graph $G(E, V)$

Output: Minimum Spanning Tree (MST)

MST = $\{\}$

for each vertex v in V :

MAKE-SET(v)

sort all edges in E in increasing order of weight

for each edge (u, v) in sorted edges:

if FIND(u) $\neq$ FIND(v):

add edge(u, v) to MST

UNION(u, v)

return MST

Code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 100
```

```
struct Edge {
```

```
    int u, v, weight;
```

```
};
```

```
struct Subset {
```

```
    int parent;
```

```
    int rank;
```

```
};
```

```
int find (struct Subset subsets[], int i) {
```

```
    if (subsets[i].parent != i) {
```

```
        subsets[i].parent = find (subsets, subsets[i].parent);
```

```
    }
```

```
    return subsets[i].parent;
```

```
}
```

void unionSet (struct Subset subsets[], int x, int y) { (17)

int rootX = find (subsets, x);

int rootY = find (subsets, y);

if (subsets[rootX].rank < subsets[rootY].rank) {

subsets[rootX].parent = rootY;

} else if (subsets[rootX].rank > subsets[rootY].rank) {

subsets[rootY].parent = rootX;

} else {

subsets[rootY].parent = rootX;

subsets[rootX].parent = rootY;

}

int compare (const void *a, const void *b) {

return ((struct Edge *) a -> weight - (struct Edge *) b -> weight);

}

void Kruskal(struct Edge edges[], int V, int E) {

struct Edge result[MAX];

struct Subset subsets[MAX];

qsort (edges, sizeof (edges[0]), compare);

for (int v = 0; v < V; v++) {

subsets[v].parent = v;

subsets[v].rank = 0;

}

int e = 0;

int i = 0;

while (e < V-1 && i < E) {

struct Edge next = edges[i++];

int x = find (subsets, next.u);

int y = find (subsets, next.v);

if (x != y) {

result[e++] = next;

unionSet (subsets, x, y);

}

10. Write a C program to demonstrate Kr

Pseudocode:

KRUSKAL(G_1):

Input: Graph $G_1(E)$

Output: Minimum

MST = $\{ \}$

for each

do {

sort edges;

for edges (u v weight) : \n";

for (int i = 0; i < E; i++) {

scanf ("%d %d", &edges[i].u, &edges[i].v,
&edges[i].weight);

}

kruskal (edges, V, E);

return 0;

}

OUTPUT:

Enter number of vertices and edges: 4 5

Enter edges (u v weight):

0 1 10

0 2 6

0 3 5

1 3 15

2 3 4

Edges

Weight

2-3

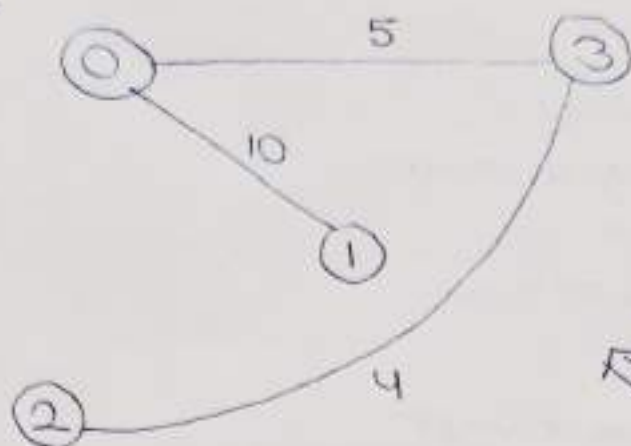
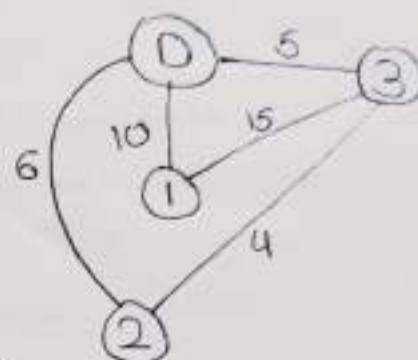
4

0-3

5

0-1

10

Graph 3Time complexity: $O(E \log E)$ MST:Graph 3

~~Y~~
25/3/26

(2)

LAP 01

Dijkstra's Algorithm

Pseudocode:

Dijkstra (Graph, source):

for each vertex v in Graph:dist[v] $\leftarrow \infty$ prev[v] $\leftarrow \text{NULL}$ dist[source] $\leftarrow 0$ $Q \leftarrow$ all vertices in Graphwhile Q is not empty: $U \leftarrow$ vertex in Q with smallest remove
 U from Q for each neighbor v of U :alt $\leftarrow$ dist[U] + weight(U, v)if alt < dist[v]:dist[v] $\leftarrow$ altprev[v] $\leftarrow U$

return dist, prev.

Code:

#include <stdio.h>

#include <limits.h>

#define MAX 100

int minDistance (int dist[], int visited[], int v) {

int min = INT_MAX, min_index = -1;

for (int i = 0; i < V; i++) {

if (!visited[i] && dist[i] < min) {

min = dist[i];

min_index = i;

}

return min_index;

3

(21)

```

void dijkstra (int graph[MAX][MAX], int V, int src) {
    int dist[MAX], visited[MAX];
    for (int i=0; i<V; i++) {
        dist[i] = INFINITY;
        visited[i] = 0;
    }
    dist[src] = 0;
    for (int count=0; count<V-1; count++) {
        int u = minDistance (dist, visited, V);
        if (u == -1) break;
        visited[u] = 1;
        for (int v=0; v<V; v++) {
            if (!visited[v] && graph[u][v] != 0 &&
                dist[u] != INFINITY && dist[u] + graph[u][v] <
                dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    printf ("Vertex id Distance from Source\n");
    for (int i=0; i<V; i++) {
        printf ("%d\t%d\n", i, dist[i]);
    }
}

int main () {
    int V, graph[MAX][MAX], src;
    printf ("Enter no. of vertices:");
    scanf ("%d", &V);
    printf ("Enter adjacency matrix:\n");
    for (int i=0; i<V; i++) {
        for (int j=0; j<V; j++) {

```

22

```
scanf("%d", &graphG[V]);
```

```
}  
}
```

```
printf("Enter source vertex (0 to %d): ", V-1);
```

```
scanf("%d", &src);
```

```
dijkstra(graph, V, &src);
```

```
return 0;
```

```
}
```

Output

Enter no. of vertices = 5

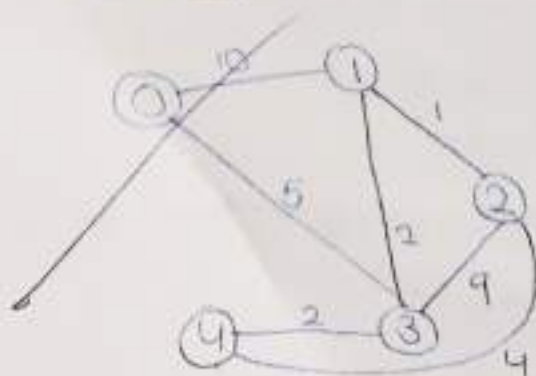
Enter adjacency matrix :

0	10	0	5	0
10	0	1	2	0
0	1	0	9	4
5	2	9	0	2
0	0	4	2	0

Enter source vertex (0 to 4): 1

vertex	distance from Source
0	7
1	0
2	1
3	2
4	4

Graph:



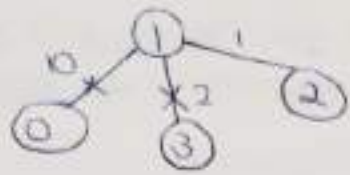
Step 0: $dist[] = [\infty, \infty, \infty, \infty, \infty]$

src=1

$visited[] = [0, 0, 0, 0, 0]$

Step 1: $U=1$ $visited[] = [0, 1, 0, 0, 0]$

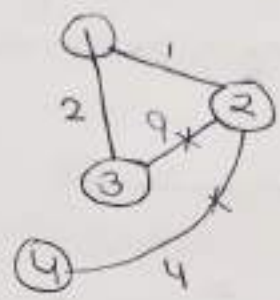
$dist[] = [10, 0, 1, 2, \infty]$



Step 2: $U=2$
(dist=1)

$visited[] = [0, 1, 1, 0, 0]$

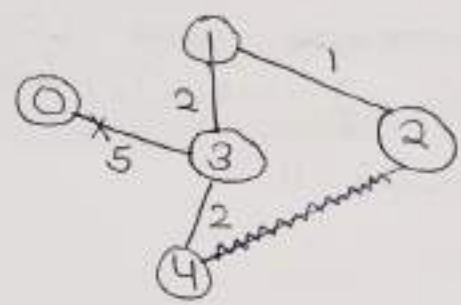
$dist[] = [10, 0, 1, 2, 5]$



Step 3: $U=3$
(dist=2)

$visited[] = [0, 1, 1, 1, 0]$

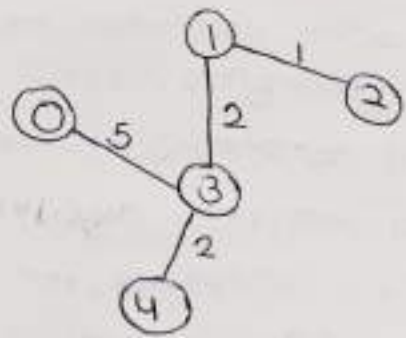
$dist[] = [7, 0, 1, 2, 4]$



Step 4: $U=4$
(dist=4)

$visited[] = [0, 1, 1, 1, 1]$

$dist[] = [7, 0, 1, 2, 4]$

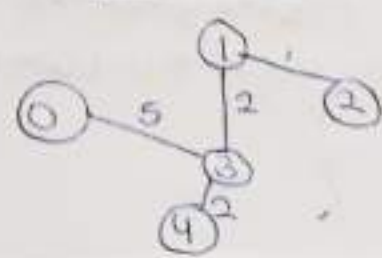


Step 5: $U=0$
(last vertex)

$visited[] = [1, 1, 1, 1, 1]$

$dist[] = [7, 0, 1, 2, 4]$

Final graph
with shortest path of ①
from all:



Handwritten signature/initials

(2)

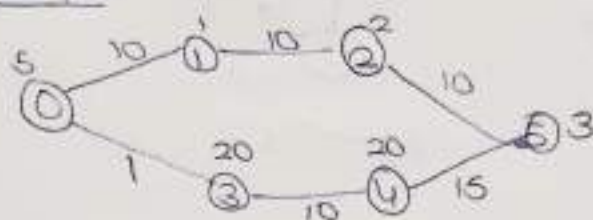
LEETCODE - 128

minimum cost to reach destination in time

There is a country of n cities numbered from 0 to $n-1$, having bi-directional roads. Roads are represented by 2D integer array edges where $edges[i] = [x_i, y_i, time_i]$

Each time on passing the city, you may pay passing fee represented by array passingFees of length n where $passingFees[j]$ is the amount of \$ you pay in city j .

In beginning, you are at city 0 & want to reach city $n-1$. In more cost of journey is the summation of passing fees. Return minimum cost

Example:

Input: maxTime = 30, edges = [[0,1,10],[1,2,10],[2,5,10],[0,3,1],[3,4,10],[4,5,15]], passingFees = [5,1,2,30,20,3]

Output: 11code:

```
int minCost(int maxTime, int** edges, int edgesSize, int* edgesColSize,
            int* passingFees, int n)
```

```
{
    int adj[n][n], costTime[n][n], deg[n],
```

```
    for (int i=0; i<n; i++) deg[i]=0;
```

```
    for (int i=0; i<edgesSize; i++){
```

```
        int u=edges[i][0], v=edges[i][1], t=edges[i][2];
```

```
        adj[u][deg[u]]=v;
```

```
        costTime[u][deg[u]++]=t;
```

```
        adj[v][deg[v]]=u;
```

```
        costTime[v][deg[v]++]=t;
```

```
    }
```

int dp[2][maxTime+1];

(25)

for (int i=0; i<n; i++) {

for (int j=0; j<=maxTime; j++) {

dp[i][j] = INF;

}

typedef struct {int node, time, cost; } state;

state heap[1000000];

int size=0;

void swap (state *a, state *b) {

state tmp = *a, *a = *b, *b = tmp;

}

void push (state val) {

int i = size++;

heap[i] = val;

while (i > 0) {

int p = (i-1)/2;

if (heap[p].cost > heap[i].cost) break;

swap (&heap[p], &heap[i]);

i = p;

}

}

state pop () {

state top = heap[0];

heap[0] = heap[--size];

int i = 0;

while (1) {

int l = 2*i, r = 2*i+2, s = 1;

if (l < size & heap[l].cost < heap[s].cost) s = l;

if (r < size & heap[r].cost < heap[s].cost) s = r;

if (s == i) break;

swap (&heap[i], &heap[s]);

i = s;

}

return top;

}

26

dp[0][0] = passingFees[0];

push((state) {0, 0, passingFees[0]});

while (size > 0) {

State cur = pop();

int u = cur.node, t = cur.time, c = cur.cost;

if (u == n-1) return c;

if (c > dp[u][t]) continue;

for (int i = 0; i < deg[u]; i++) {

int v = adj[u][i];

int nt = t + costTime[u][i];

if (nt > maxTime) continue;

int nc = c + passingFees[v];

if (dp[v][nt] > nc) {

dp[v][nt] = nc;

push((state) {v, nt, nc});

}

}

}

return -1;

}

Directed output : 11

Output: 11

11/4/26

Q. Implement Fractional Knapsack problem using Greedy Technique

[Pseudocode]

FractionalKnapsack (items, capacity):

Each item has:

value[i], weight[i]

n ← number of items

Step1: compute value per unit weight

for i ← 1 to n:

ratio[i] ← value[i] / weight[i]

Step2: Sort items in descending order of ratio

(Sort items by ratio in descending order)

totalvalue ← 0

Step3: Take items greedily

for i ← 1 to n:

if weight[i] ≤ capacity:

Take full item

totalvalue ← totalvalue + value[i]

capacity ← capacity - weight[i]

else

Take fraction of item

fraction ← capacity / weight[i]

totalvalue ← totalvalue + value[i] * fraction

capacity ← 0

break

return totalvalue

Ex-

capacity = 100

	A	B	C	D	E	F
Weight:	20	30	40	100	150	200
Value:	350	100	150	200	250	300

(V/w) ratio: 17.5 3.3 3.75 2 1.66 1.5

Ans: op:

	A	C	B	D	E	F
wt	20	40	20	100	150	200
wt-cap	wt-cap	wt-cap	wt-cap			

$$\text{totalvalue} = 350 + 150 + 100 + \frac{10}{100} \times 200 = 620$$

(26)

C++

#include <stdio.h>

struct Item {

int weight;

int value;

};

void swap (struct Item *a, struct Item *b) {

struct Item temp = *a;

*a = *b;

*b = temp;

}

void sortItems (struct Item arr[], int n) {

for (int i = 0; i < n-1; i++) {

for (int j = 0; j < n-i-1; j++) {

float r1 = (float) arr[j].value / arr[j].weight;

float r2 = (float) arr[j+1].value / arr[j+1].weight;

if (r1 < r2) {

swap (&arr[j], &arr[j+1]);

}

float fractionKnapsack (struct Item arr[], int n, int capacity,

sortItems (arr, n);

float datavalue = 0.0;

for (int i = 0; i < n; i++) {

if (capacity >= arr[i].weight) {

capacity -= arr[i].weight;

datavalue += arr[i].value;

} else {

float fraction = (float) capacity / arr[i].weight;

datavalue += arr[i].value * fraction;

break;

}

}

return datavalue;

```
int main() {
```

```
    int n, capacity;
```

```
    printf ("Enter number of items : ");
```

```
    scanf ("%d", &n);
```

```
    struct Item arr[n];
```

```
    printf ("Enter value and weight of each item : \n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf ("%d %d", &arr[i].value, &arr[i].weight);
```

```
    }
```

```
    printf ("Enter knapsack capacity : ");
```

```
    scanf ("%d", &capacity);
```

```
    float maxvalue = fractionalKnapsack (arr, n, capacity);
```

```
    printf ("Maximum value in knapsack = %.2f \n", maxvalue);
```

```
    return 0;
```

```
}
```

OUTPUT:

Enter number of items : 6

Enter value and weight of each item :

350 20

100 30

100 40

200 100

250 150

300 200

Enter knapsack capacity : 100

Maximum value in knapsack : 620

(3)

LeetCode - 322

Coin Change

Q. You are given with an integer array coins representing coins of different denominations and an integer amount representing a total amount of money.

Return the fewest number of coins that you need to make up that amount. If no combination exists, return -1.

C++

```
int coinChange (int coins[], int n, int amount) {
```

```
    int dp[amount+1];
```

```
    for (int i=0; i<=amount; i++) {
```

```
        dp[i] = amount+1;
```

```
    }
```

```
    dp[0] = 0;
```

```
    for (int i=1; i<=amount; i++) {
```

```
        for (int j=0; j<n; j++) {
```

```
            if (coins[j] <= i) {
```

```
                if (dp[i-coins[j]] + 1 < dp[i]) {
```

```
                    dp[i] = dp[i-coins[j]] + 1;
```

```
            }
```

```
        }
    }
```

```
    return (dp[amount] > amount) ? -1 : dp[amount];
```

Testcase -

Input: coins = [1, 2, 5]

amount = 11

Output

3 ✓

Expected

3 ✓

⇒ Implement all pair shortest Path's problem using

Floyd's Algorithm

Pseudocode -

Floyd (W/n):

W is the weight matrix of size $n \times n$

n is the number of vertices

D = W # Initialize distance matrix

for k = 1 to n:

for i = 1 to n:

for j = 1 to n:

if $D[i][j] > D[i][k] + D[k][j]$:

$D[i][j] = D[i][k] + D[k][j]$

return D.

Code -

```
#include <stdio.h>
```

```
#define INF 9999
```

```
int main() {
```

```
    int V;
```

```
    printf("Enter the number of vertices ");
```

```
    scanf("%d", &V);
```

```
    int graph[V][V];
```

```
    printf("Enter the adjacency matrix: \n");
```

```
    printf("Enter %d for INF if no edge exists)\n", INF);
```

```
    for (int i = 0; i < V; i++) {
```

```
        for (int j = 0; j < V; j++) {
```

```
            scanf("%d", &graph[i][j]);
```

```
        }
```

```
    int dist[V][V];
```

```
    for (int i = 0; i < V; i++) {
```

```
        for (int j = 0; j < V; j++) {
```

```
            scanf("%d", &dist[i][j]) dist[i][j] = graph[i][j];
```

```
        }
```

32)

```
for (int k=0; k<V; k++) {
```

```
    for (int i=0; i<V; i++) {
```

```
        for (int j=0; j<V; j++) {
```

```
            if (dist[i][k] != INF && dist[k][j] != INF &&
```

```
                dist[i][j] > dist[i][k] + dist[k][j]) {
```

```
                dist[i][j] = dist[i][k] + dist[k][j];
```

```
        }
```

```
    }
```

```
    for (int i=0; i<V; i++) {
```

```
        for (int j=0; j<V; j++) {
```

```
            if (dist[i][j] == INF) {
```

```
                printf("%7s", "INF");
```

```
            } else {
```

```
                printf("%7d", dist[i][j]);
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

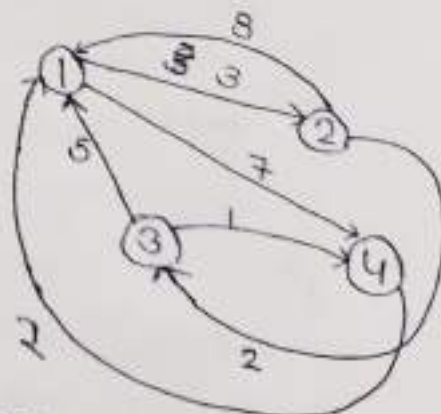
OUTPUT:

Enter number of vertices: 4

Enter the adjacency matrix:

(Enter 99999 for INF if no edge exists)

```
0
3
99999
7
8
0
2
99999
0
1
2
99999
99999
0
```

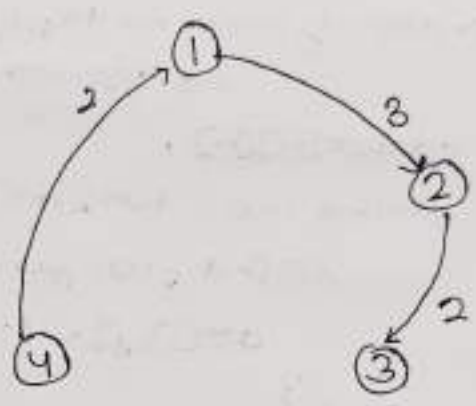
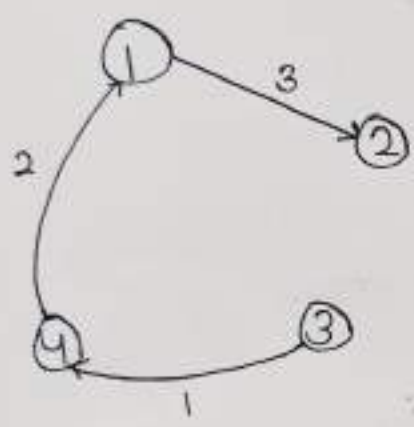
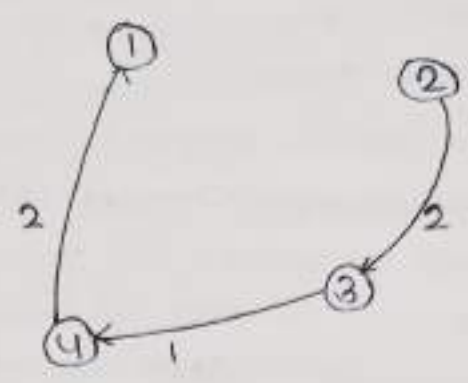
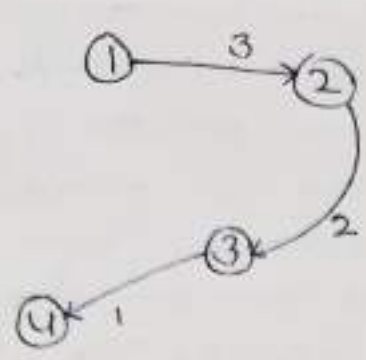


Shortest distance matrix:

0	3	5	6
5	0	2	3
3	8	0	1
2	5	7	0

24/1/26

Graphs



[Handwritten signature]

(34)

LEETCODE - 1334

- ① There are n cities numbered from 0 to $n-1$. Given the edges where $\text{edges}[i] = [\text{from}_i, \text{to}_i, \text{weight}_i]$ represents a bidirectional & weighted edge b/w cities $\text{from}_i, \text{to}_i$ & the integer distanceThreshold . Return the city with the smallest number of cities that are reachable through some path & whose distance is at most distanceThreshold . If there are multiple such cities, return the city with the greatest number.

Code:

```
int findTheCity(int n, int** edges, int edgesSize, int* edgesCol,
               int distanceThreshold) {
    int dist[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = (i == j) ? 0 : 10001;
        }
    }
    for (int i = 0; i < edgesSize; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        int w = edges[i][2];
        if (w <= distanceThreshold) {
            dist[u][v] = w;
            dist[v][u] = w;
        }
    }
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}
```

```

int minReachableCount = n;
int resultCity = -1;
for (int i = 0; i < n; i++) {
    int reachableCount = 0;
    for (int j = 0; j < n; j++) {
        if (i != j && distG[i][j] <= distanceThreshold) {
            reachableCount++;
        }
    }
    if (reachableCount <= minReachableCount) {
        minReachableCount = reachableCount;
        resultCity = i;
    }
}
return resultCity;

```

OUTPUT:

Input

n=4

edges = [[0,1,3], [0,2,1], [0,3,4], [2,3,1]]

distanceThreshold = 4

Output: 3

Expected: 3


24/4/20

(36)

LAB-07

Q. Implement 0/1 knapsack problem using dynamic programming

Pseudocode:

Function knapsack(wt, value, n, W):

create dp[0..n][0..W]

For w=0 to W:

dp[0][w] = 0

For i=0 to n:

dp[i][0] = 0

For i=1 to n:

For w=1 to W:

if wt[i-1] ≤ w then

include = val[i-1] + dp[i-1][w-wt[i-1]]

exclude = dp[i-1][w]

dp[i][w] = max(include, exclude)

else

dp[i][w] = dp[i-1][w]

return dp[n][W]

Code:

```
#include <stdio.h>
```

```
int max (int a, int b) {
```

```
    return (a>b) ? a : b;
```

```
}
```

```
int knapsack (int W, int wt[], int val[], int n) {
```

```
    int dp[n+1][W+1];
```

```
    for (int i=0; i<=n; i++) {
```

```
        for (int w=0; w<=W; w++) {
```

```
            if (i==0 || w==0) {
```

```
                dp[i][w] = 0;
```

```
            }
```

else if (wt[i-1] <= W) {

int include = val[i-1] + dp[i-1][W-wt[i-1]];

int exclude = dp[i-1][W];

dp[i][W] = max(include, exclude);

}

else {

dp[i][W] = dp[i-1][W];

}

}

}

return dp[n][W];

}

int main () {

int n, W;

printf ("Enter no. of items : ");

scanf ("%d", &n);

int val[n], wt[n];

for (int i=0; i<n; i++) {

scanf ("%d", &val[i]);

}

printf ("Enter weight of items : ");

for (int i=0; i<n; i++) {

scanf ("%d", &wt[i]);

}

printf ("Enter the capacity of knapsack : ");

scanf ("%d", &W);

int result = knapsack (W, wt, val, n);

printf ("maximum value in knapsack = %d\n",
result);

return 0;

}

38

OUTPUT

Enter number of items : 4

Enter values of items :

15

20

10

30

Enter weight of items :

2

3

1

4

Enter capacity of knapsack = 5

Maximum value in knapsack = 40

Tracing

initially

n/w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					

item1:

n/w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	15	15	15	15
2	0					
3	0					
4	0					

item2

n/w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	15	15	15	15
2	0	0	15	20	20	20
3	0					
4	0					

item3:

n/w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	15	15	15	15
2	0	0	15	20	20	20
3	0	10	15	20	30	35
4	0					

item4:

n/w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	15	15	15	15
2	0	0	15	20	20	20
3	0	10	15	20	30	35
4	0	10	15	20	30	(40)

knapsack: $\{1, 0, 1\} \rightarrow \text{max value in knapsack} = 40$

(6)

Leetcode - 638

In a store, there are n items to sell. Each item has a price. However, there are some special offers, and a special offer consists of one or more different kinds of items with a sale price.

You are given an integer array `price` where `price[i]` is the price of the i th item and an integer array `needs` where `needs[i]` is the number of pieces of the i th item you want to buy. You are also given an array `special` where `special[i]` is of size $n+1$ where `special[i][j]` is the number of pieces of the j th item in the i th offer and `special[i][n]` is the price of i th offer. Return the lowest price you have to pay for `needs`. You are not allowed to buy more items than you want, even if that would lower the overall price. You could use any of the special offers as many times as you want.

Solution:

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

```
static int dfs(int* price, int priceSize, int** special, int specialSize,
               int* needs, int* memo, int* base) {
```

```
    int key = 0;
```

```
    for (int i = 0; i < priceSize; i++) {
```

```
        key = key * 11 + needs[i];
```

```
    } if (memo[key] != -1) return memo[key];
```

```
    int minCost = 0;
```

```
    for (int i = 0; i < priceSize; i++) {
```

```
        minCost += needs[i] * price[i];
```

```
    }
```

```
    for (int i = 0; i < specialSize; i++) {
```

```
        int valid = 1;
```

```
        for (int j = 0; j < priceSize; j++) {
```

```
            if (needs[j] < special[i][j]) {
```

```
                valid = 0;
```

```
            } break;
```

3

if (valid) {

for (int j=0; j<priceSize; j++) {

needs[j] -= special[i][j];

3

int offornst = special[i][priceSize] + dfs(price, priceSize,
special, specialSize, needs, memo, base);

for (int j=0; j<priceSize; j++) {

needs[j] += special[i][j];

3

3

3

memo[key] = minCost;

return minCost;

3

int shoppingOffers (int* price, int priceSize, int** special, int
specialSize, int* specialColSize, int* needs, int
needsSize) {

int maxStates = 1;

for (int i=0; i<priceSize; i++) maxStates *= 11;

int* memo = (int*) malloc (maxStates * sizeof (int));

for (int i=0; i<maxStates; i++) memo[i] = -1;

int result = dfs (price, priceSize, special, specialSize, needs,
memo, needs);

free (memo);

return result;

3

#ifdef LOCAL_TEST

int main() {

int price[] = {2, 5};

int priceSize = 2;

int specialArr[][3] = {{3, 0, 5}, {1, 2, 10}};

int specialSize = 2;

int* special[2];

int specialColSize[] = {3, 3};

Vas


```

(12) for(int i=0; i<specialSize; i++){
    special[i] = specialOffer[i];
}
int needs[] = {3,2};
int needsSize = 2;
int result = shoppingOffers(price, priceSize, special, specialSize,
    needs, needsSize);
printf("Minimum cost: %d\n", result);
return 0;
}
// end of.

```

OUTPUT:

Input:

Price = [2,5]

special = [[3,0,5], [1,2,10]]

needs = [0,2]

Output

14

Expected

14

Topological ordering of vertices in a given diagram

Pseudocode

Topological-sort (G)

Input

Directed Graph $G(V, E)$

Output

Topological ordering of vertices

Create an empty stack

Mark all vertices as unvisited

for each vertices V in G do

if V is unvisited then

DFS(V)

while S is not empty do

print pop(S)

DFS(V)

mark as visited

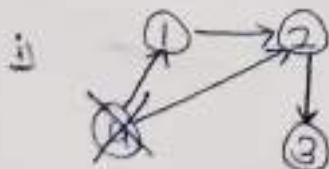
for each adjacent vertex U of V do

if U is unvisited then

DFS(U)

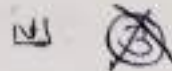
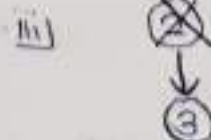
Push V onto stack S

Example



Sorting:

4			
---	--	--	--



[Signature]

Sorting:

4	1		
---	---	--	--

4	1	2	
---	---	---	--

4	1	2	3
---	---	---	---

44

code:

#include <stdio.h>

#define MAX 100

int stack[MAX];

int top = -1;

int visited[MAX];

int graph[MAX][MAX];

int n;

void push(int v)

{

stack[++top] = v;

}

void DFS(int v)

{

visited[v] = 1;

for (int i = 0; i < n; i++)

{

if (graph[v][i] == 1 && !visited[i])

{

DFS(i);

}

}

push(v);

}

void topologicalSort()

{

for (int i = 0; i < n; i++)

{

visited[i] = 0

}

for (int i = 0; i < n; i++) {

if (!visited[i])

{

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

DFS(i);

(46)

}

}

printf ("Topological Ordering:");

while (top != -1) {

printf ("%d", stack[top--]);

}

}

int main () {

{

int edges, u, v;

printf ("Enter no. of vertices:");

scanf ("%d", &n);

printf ("Enter no. of edges:");

scanf ("%d", &edges);

for (int i=0; i<n; i++)

for (int j=0; j<n; j++) {

graph[i][j] = 0;

printf ("Enter edges (u v): ");

for (int i=0; i<edges; i++)

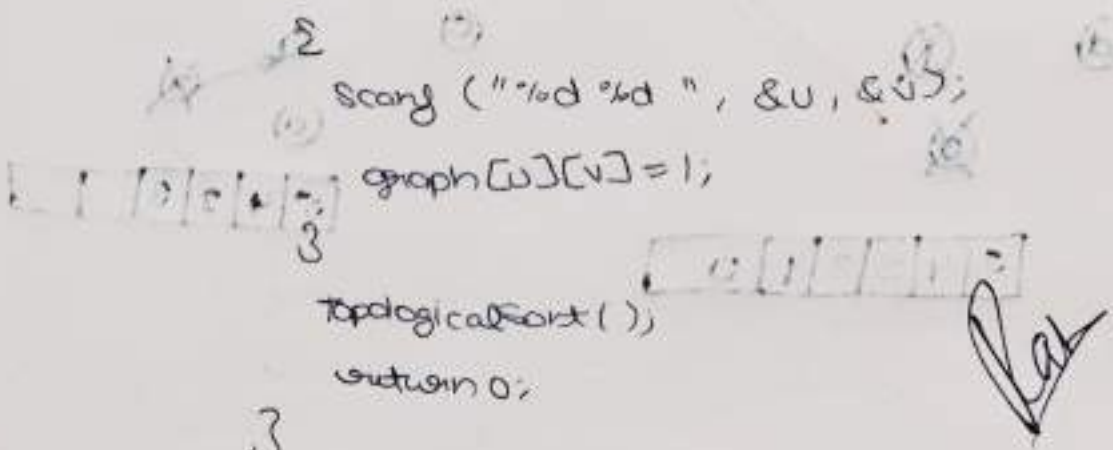
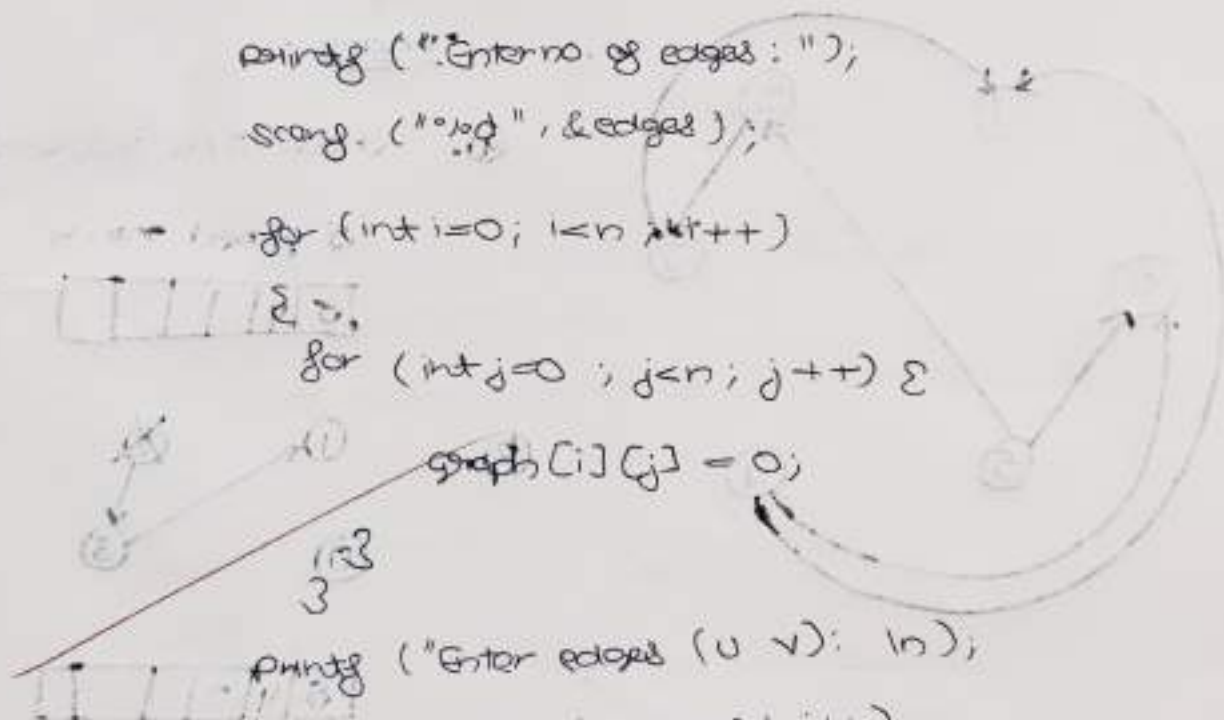
scanf ("%d %d", &u, &v);

graph[u][v] = 1;

topologicalSort();

return 0;

}



46

Tracing Output:

Enter no of vertices : 6

Enter no of edges : 6

Enter edges (u v):

5 2

5 0

4 0

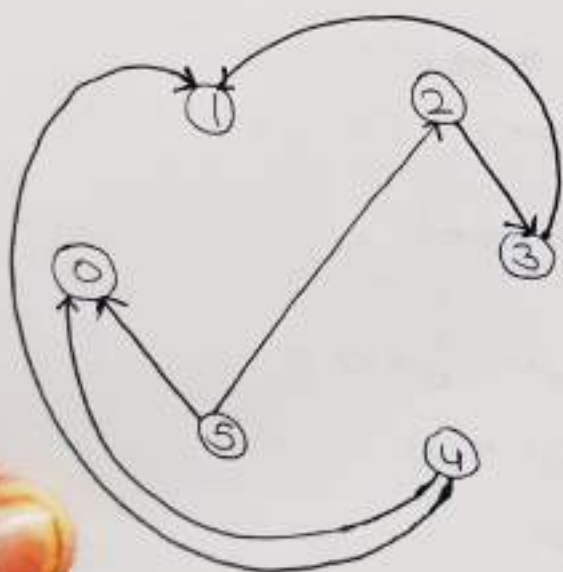
4 1

2 3

3 1

Output: Topological ordering :

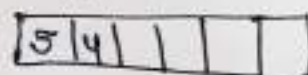
5 4 2 3 1 0



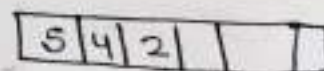
Tracing:

a. chose 5/4 indegree=0

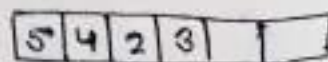
5 picked then 4



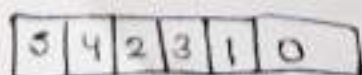
b.



c.



d.



"N-Queens Problem" using Backtracking

Pseudocode

N_QUEENS (row, N)

if row > N then

print solution

return

for col = 1 to N do

if Safe (row, col) then

place queen at (row, column)

N_QUEENS (row + 1, N)

remove queen from (row, col)

Safe (row, col)

for i = 1 to row - 1 do

if queen is placed at (i, col)

return false

if queen is placed at (i, col - (row - i))

return false

if queen is placed at (i, col + (row - i))

return false

return true.

code:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int board [MAX];
```

```
int safe (int row, int col)
```

```
{
```

```
for (int i = 0; i < row; i++)
```

```
{
```

Kab

48

```
if (board[i] == col)
```

```
return 0;
```

```
if ((board[i]-1) == (col+row) ||
```

```
(board[i]+1) == (col+row))
```

```
return 0;
```

```
}
```

```
return 1;
```

```
}
```

```
void NQueens(int row, int n)
```

```
{
```

```
if (row > n)
```

```
{
```

```
printf("N solution: \n");
```

```
for (int i = 1; i <= n; i++)
```

```
{
```

```
for (int j = 1; j <= n; j++)
```

```
{
```

```
if (board[j] == j)
```

```
printf("Q");
```

```
else
```

```
printf(".");
```

```
}
```

```
return;
```

```
}
```

```
for (int col = 1; col <= n; col++)
```

```
{
```

```
if (safe(row, col))
```

```
{
```

```
board[row] = col;
```

```
NQueens(row+1, n);
```

```
board[row] = 0;
```

```
}
```

```
}
```

```
}
```

```
int main()
{
```

```
    int n;
```

```
    printf("Enter no. of queens: ");
```

```
    scanf("%d", &n);
```

```
    N-Queens(1, n);
```

```
    return 0;
```

3

OUTPUT:

Enter no. of queens = 4

Solution:

```

. Q . .
. . . Q
Q . . .
. . Q .

```

```

. . Q .
Q . . .
. . . Q
. Q . .

```

Johnson-Trotter AlgorithmPseudocode:

Algorithm JohnsonTrotter (N):

Initialize an array A of size N with values 1 to N
 " " " 0 " " " left direction

while there exists at least one mobile element in A do

 largest-mobile = -1

 mobile-index = -1

 for i from 1 to N do

 if isMobile(A, 0, i, N) and $A[i] > \text{largest-mobile}$:

 largest-mobile = $A[i]$;

 mobile-index = i

 if $D[\text{mobile-index}] == \text{'left'}$ then

 neighbor-index = mobile-index - 1

 else

 neighbor-index = mobile-index + 1

 swap ($A[\text{mobile-index}]$, $A[\text{neighbor-index}]$)

 swap ($D[\text{mobile-index}]$, $D[\text{neighbor-index}]$)

 for i from 1 to N do

 if $A[i] > \text{largest-mobile}$ then:

 if $D[i] == \text{'left'}$ then:

$D[i] = \text{'right'}$;

 else

$D[i] = \text{'left'}$

Print / Output current permutation A

Function IsMobile (A, D, index, N)

if $D[\text{index}] == \text{'left'}$ & $\text{index} > 1$ then

 if $A[\text{index}] > A[\text{index}-1]$ then

 return true

9
if D[index] == 'right' and index < N then:

if A[index] > A[index+1] then

return true

end function

code:

```
#include <stdio.h>
```

```
#define Left -1
```

```
#define Right 1
```

```
int getMobile (int a[], int dir[], int n)
```

```
{
```

```
int mobile = 0, mobile_prev = 0;
```

```
for (int i = 0; i < n; i++)
```

```
{
```

```
if (dir[a[i]-1] == Left && i != 0)
```

```
{
```

```
if (a[i] > a[i-1] && a[i] > mobile_prev)
```

```
{
```

```
mobile = a[i];
```

```
mobile_prev = mobile;
```

```
}
```

```
if (dir[a[i]-1] == Right && i != n-1)
```

```
{
```

```
if (a[i] > a[i+1] && a[i] > mobile_prev)
```

```
{
```

```
mobile = a[i];
```

```
mobile_prev = mobile;
```

```
}
```

```
return mobile;
```

```
}
```

```
void solveTraveller (int n)
```

```
{
```

```
int a[n], dir[n];
```

Lab

52

```
for (int i=0; i<n; i++)
```

```
{
```

```
    a[i] = i+1;
```

```
    dir[i] = left;
```

```
}
```

```
printReimutation(a, n);
```

```
while (1) {
```

```
    int mobile = getMobile(a, dir, n);
```

```
    if (mobile == 0)
```

```
        break;
```

```
    int pos;
```

```
    for (int i=0; i<n; i++) {
```

```
        if (a[i] == mobile) {
```

```
            pos = i;
```

```
            break;
```

```
        }
```

```
    if (dir[pos] == left) {
```

```
        int temp = a[pos];
```

```
        a[pos] = a[pos-1];
```

```
        a[pos-1] = temp;
```

```
        pos = pos-1;
```

```
    }
```

```
    else {
```

```
        int temp = a[pos];
```

```
        a[pos] = a[pos+1];
```

```
        a[pos+1] = temp;
```

```
        pos = pos+1;
```

```
    }
```

```
    for (int i=0; i<n; i++)
```

```
    {
```

```
        if (a[i] > mobile) {
```

```
            dir[a[i]-1] = -dir[a[i]-1];
```

```
        }
```


print Permutation(a,n);

(53)

3
3

int main () {

int n;

printf("Enter value of n:");

scanf("%d", &n);

johnsonTrotter(n);

return 0;

}

OUTPUT :

Enter value of n: 3

1 2 3

1 3 2

3 1 2

3 2 1

2 3 1

2 1 3

(2/2)

Pseudocode:

```

HeapSort(A, n)
  for i ←  $\lfloor n/2 \rfloor$  down to 1 do
    Heapify(A, n, i)
  for i ← n down to 2 do
    swap(A[i], A[1])
    Heapify(A, i-1, 1)

Heapify(A, size, i)
  largest ← i
  left ←  $2*i$ 
  right ←  $2*i + 1$ 
  if left ≤ size and A[left] > A[largest]
    largest ← left
  if right ≤ size and A[right] > A[largest]
    largest ← right
  if largest ≠ i
    swap(A[i], A[largest])
    Heapify(A, size, largest)

```

Code:

```

#include <stdio.h>

void heapify (int a[], int n, int i)
{
  int largest = i;
  int left =  $2*i + 1$ ;
  int right =  $2*i + 2$ ;
  if (left < n && a[left] > a[largest])
    largest = left;
  if (right < n && a[right] > a[largest])
    largest = right;
  if (largest != i)
  {
    int temp = a[i];
    a[i] = a[largest];
    a[largest] = temp;
  }
}

```

heapify (a, n, largest);

}

}

void heapsort (int a[], int n)

{

for (int i = n/2 - 1; i >= 0; i--)

heapify (a, n, i)

for (int i = n - 1; i > 0; i--)

{

int temp = a[0];

a[0] = a[i];

a[i] = temp;

heapify (a, i, 0);

}

int main()

{

int n;

printf ("Enter no. of elements:");

scanf ("%d", &n);

int a[n];

printf ("Enter elements:");

for (int i = 0; i < n; i++)

scanf ("%d", &a[i]);

heapsort(a, n);

printf ("Sorted array:");

for (int i = 0; i < n; i++)

printf ("%d", a[i]);

return 0;

}

56

OUTPUT :

Enter the no. of elements : 5

Enter elements : 40

34

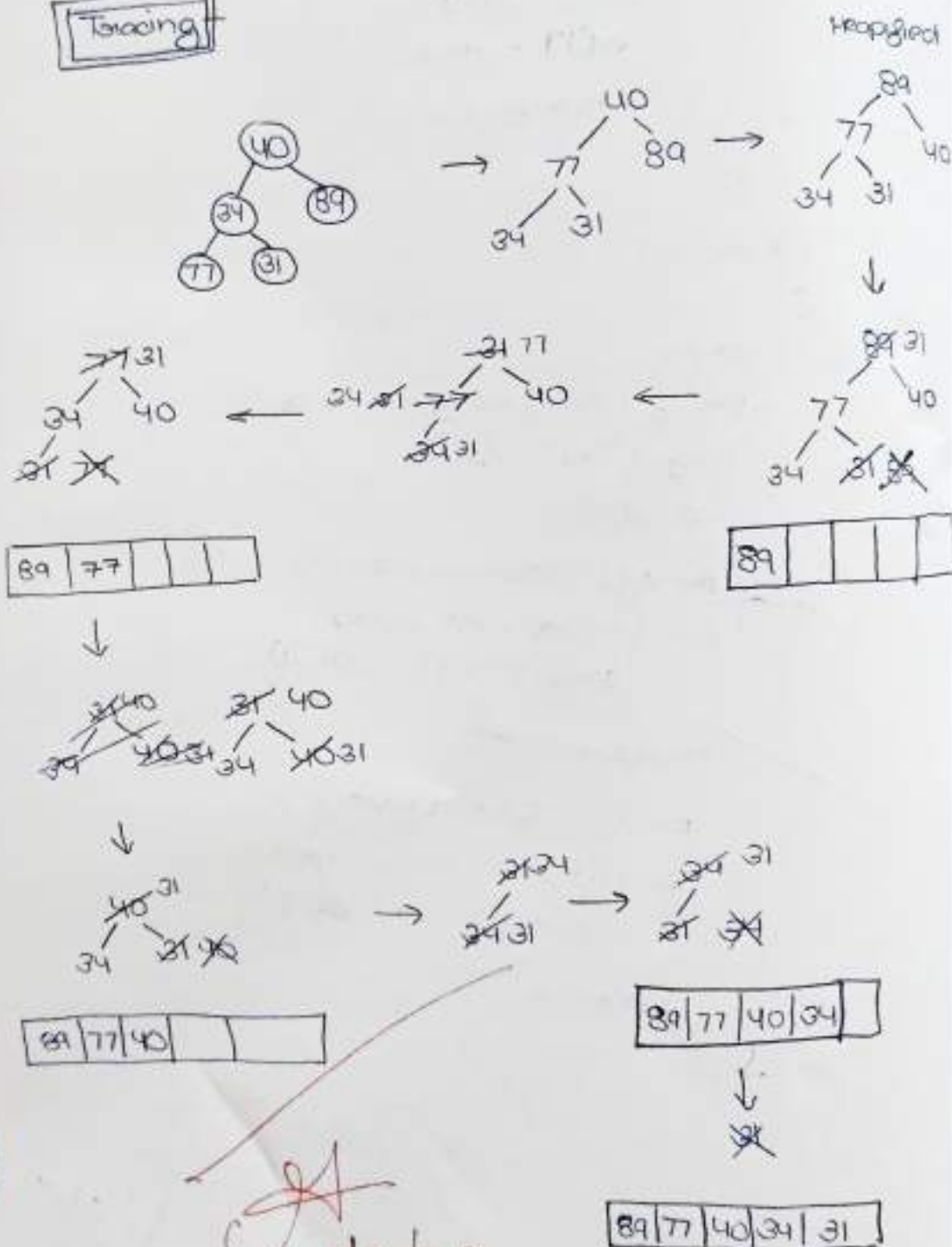
89

77

31

Sorted array : 31 34 40 77 89

Tracing



9/6/20